Web Agent 的并行执行与容错调度

陈思旭　寇之洲　谢树韬

第 7 队　勇攀高峰

代码仓库：`github.com/xie-st/WebAgentNew`

摘要：针对网页任务中的连续交互、结果确认和容错调度问题，本文基于 EffGen 构建统一的网页智能体执行框架，通过可暂停的生成器连接官方评测平台与 Playwright 浏览器。最终官方配置采用单 Agent 串行执行，启用任务状态工具与基于页面证据的完成核验，将不合法动作和核验缺口反馈给同一执行循环。恢复与接管、必要上下文保护、批准文本经验和只读并行调度作为独立配置或扩展模块保留。工程过程由人工规划、文档约束、智能体分工与统一集成组成，形成可追溯的代码、配置和运行记录。

关键词：网页智能体；结果核验；状态管理；容错调度；并发控制

1　问题定义与基础工具

本项目面向网页任务的并行执行、结果确认、容错恢复与频率控制。比赛运行模型为 Qwen3.6-35B-A3B，官方端通过 SDK 获取页面观察、调用统一模型并返回单个网页动作，真实端由 Playwright 驱动浏览器渲染、操作与截图。系统以 EffGen 为基础，复用其观察—思考—工具调用—反馈循环，以及轨迹、输入预算和历史压缩机制。[1]

开发工具采用 Codex、WorkBuddy 和 DeepSeek Harness（DSH），提供文件读写、代码检索、命令执行、测试和任务派发能力。Codex 使用 GPT-6.1 Sol，推理强度为 Medium；WorkBuddy 使用 GLM-5.3-flash；DSH 经硅基流动 API 调用 DeepSeek v4 flash 1013。这些开发工具提供通用工程能力，不包括本项目所需的 Web Agent 具体知识；赛题理解、论文选择与路线规划由团队完成。

2　工程过程与分工

团队先确定赛题约束与优化方向，查询状态维护、观察压缩、完成核验和并发调度等论文，形成 RoadMap。Codex 将其翻译为阶段方案、接口约定、文件归属和验收要求；人工审定后，WorkBuddy 向 DSH 派发任务，主线程统一集成并核对实际入口。工程先接通两套环境，再建设状态、核验、恢复与调度能力，最后按主树入口和配置收敛提交方案。

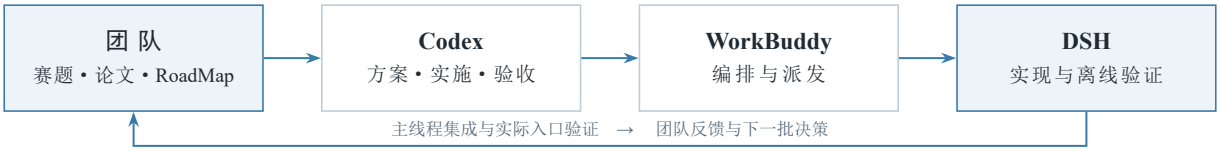


图 1　人机协作的工程开发流程

并行开发沿用模块边界。输入与观察适配、状态与恢复支持、本地任务与判定分别在独立源码副本中实现；运行测试前确认导入各自副本。公共驱动、官方入口、配置和命令行由一名负责人顺序合入，跨模块功能在接口稳定后连接。真实模型与官方平台任务顺序执行，开发智能体的并行不改变比赛执行端的并发约束。

3 最终官方执行方案

3.1 统一接口与可暂停循环

最终入口为 `webagent.official_agent.OfficialAgent`，提交配置为 `code/harness/configs/official.yaml`。每个任务建立独立 Session、状态库、模型适配器和 EffGen 驱动。官方端模型调用仅经过 SDK；统一网页工具向后端交付点击、输入、按键、滚动、导航等动作，模型依据返回的可访问性树（AX）继续决策。



图 2 共用决策循环的双环境接入结构

网页动作处通过生成器暂停，下一次 `step()` 将观察送回原循环；状态读取、计划更新和证据记录等内部工具直接本地执行。动作绑定页面、观察版本和当前有效元素编号，错误目标或不合法参数在发送前拒绝，并通过工具反馈继续纠正，默认最多允许两次格式纠正。

输入采用 `decompose` 语义：多行文本拆成输入与 Enter 子动作，每个子动作接收新的页面反馈，完整序列结束后再交回模型；空文本提交转换为 Enter，空文本且不提交的动作直接反馈纠正。文本替换由明确的选中、清除和输入动作完成，保留真实浏览器输入语义。

算法 1 最终官方入口的连续执行流程

```
建立任务预算、状态库和会话，注册启用工具，安装首个观察
while 任务活动且未到平台截止时间：
    推进同一 EffGen 生成器，取得提案 p
    if p 是内部工具：本地执行并反馈；continue
    if p 请求完成：核验已声明目标；通过则结束
        否则返回缺口继续修正，超出限额则记为 incomplete
    else 校验观察版本与动作目标；不合法则反馈纠正
        必要时拆分输入；逐个交出动作，接回新观察和反馈
        保存动作与证据，推进原循环
保存终态、核验结果、请求记录与终止原因
```

3.2 状态账本与完成核验

默认注册 `goal_declare`、状态读写和计划更新工具。模型声明业务要求及可观察的完成条件，程序绑定当前版本、分支和证据来源。状态库保存目标、事实、依赖与证据引用，更新时校验版本，再将完整 JSON 快照原子替换；截图与原始观察沿用证据存储，不在账本中重复保存。

完成判定检查已声明目标。官方路径使用确定性条件核对当前 URL、AX 文本、字段值和已保存证据，核验器自身不执行网页动作。`request_finish` 未通过时，把未满足的目标与条件反馈给同一生成器，允许两次继续修正，超过限额则以 `incomplete` 结束。字段值核验适配官方 AX 的属性后缀和子级文本，区分操作意图与实际页面结果。

3.3 提交配置与预算

下表依据当前主树配置及其实际解析结果。状态工具与完成核验进入默认执行路径；其余扩展通过独立开关控制。

表 1 主树最终官方配置

项目	最终官方配置
决策与执行	单 Agent、文本 ReAct；同一时刻仅一个任务、一个动作和一次模型调用；关闭自动子智能体、推测执行和跨任务记忆。
默认能力	`state_tools=true`，`verification=true`。
可选开关	`recovery=false`，`context.enabled=false`，`experience.enabled=false`，`parallel.enabled=false`。
输入预算	原生压缩输入预算 24,000 token，规划上下文 32,768 token；采用估算计数。
请求约束	模型出口检查最多 256 条消息、1 MiB JSON 请求；必要任务与当前观察不因字节超限而静默删除。
截止与终态	提前截止余量为 0，遵守平台绝对截止时间；终止时保存结果、调用记录和错误原因。

4 可选机制与并发调度

4.1 恢复、接管与上下文

开启恢复开关后，官方入口连接故障分类、停滞检测和有界恢复。恢复依据失败证据与目标依赖撤回受影响的完成判断，保留其他已确认进度；每目标最多两次、全任务最多五次恢复。待确认副作用在动作发出前入账，反馈未知时保留未知状态，先核实再决定后续动作。

同一开关接入单任务人工接管：登录、验证或主动暂停时关闭动作出口，处理已执行反馈后等待明确继续信号；恢复时重新观察，废弃旧提案与旧元素编号。快照重启恢复业务状态与证据，并重新建立执行循环。

必要上下文保护开启后，在模型发送前核对任务、当前 AX、账本短视图及待确认副作用，超限时明确拒绝发送。经验加载开启后，仅从本地批准清单选择版本与适用条件匹配的文本，进入既有提示词装配；当前提交配置关闭这两项，默认执行沿用 EffGen 原生上下文管理。

4.2 只读并行与统一限流

调度模块通过独立协调器接收分支任务，每个分支关联会话、循环和页面槽位；仅独立只读分支可并行，计划中出现写任务时整批退回串行。所有分支共用模型调用门控、访问节奏与根预算，失败、取消和丢弃分支同样计入汇总。模型并发固定为一，站点操作间隔默认一秒，遇到冷却提示时共享退避状态。

当前正式官方入口采用串行路径，配置拒绝开启多 Tab；独立调度模块保留给真实端扩展。并行模块的多分支暂停、继续与排空仍是预留接口，单任务恢复与接管由官方入口中的控制层承担。

4.3 论文方法与方案取舍

论文为机制选择提供依据，具体接入以当前代码和配置为准。状态、核验与统一执行进入官方主流程，其他方法按下表保留相应模块或候选形式。

表 2 论文方法与最终方案的对应

方向	论文依据	本项目采用方式
状态与停滞	PoS / BeliefState [2]	账本区分事实和目标缺口；停滞诊断随恢复开关启用。
核验与纠错	WALT [3]、Reflexion [4]	默认核验已声明目标，反馈送回同一循环；有界恢复按证据撤回受影响状态。
摘要与经验	M² [5]	提供批准文本经验加载，默认关闭；同轮摘要保留为候选。
历史管理	AgentProg [6]、Read More, Think More [7]	复用原生轨迹与压缩；子目标折叠、历史差分保留为候选。
页面筛选	Prune4Web [8]	关键词与固定程序评分保留为候选，当前主流程读取 AX。
依赖调度	Flash-Searcher [9]	独立协调器实现只读分支调度、共享写退回串行及根预算汇总。
视觉与行动效率	UI-TARS [10]、WebClipper[11]	截图用于留证；截图参与决策、轨迹裁剪及动作组合保留为候选。

5 工程交付与追溯

提交工程包含 `code/harness/`、固定版本及补丁的 EffGen、官方 SDK 和配置。两套后端共用观察、动作、会话与证据协议；日志关联任务、页面、动作反馈、模型请求和终止原因。官方请求的真实 token 用量以平台记录为准，本地估算值单独标明。最终方案以本次读取的主树源码和配置为依据，相关文件哈希保存在报告来源目录，便于材料与工程版本核对。

参考文献

[1] EffGen: Enabling Small Language Models as Capable Autonomous Agents. arXiv:2602.00887, 2026.

[2] Beyond Memory: Harnessing Long-Horizon Agents with Explicit Belief States. arXiv:2610.01415, 2026.

[3] WALT: Web Agents that Learn Tools. arXiv:2510.01524, 2025.

[4] Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366, 2023.

[5] M²: Dual-Memory Augmentation for Long-Horizon Web Agents via Trajectory Summarization and Insight Retrieval. arXiv:2603.00503, 2026.

[6] AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. arXiv:2512.10371, 2025.

[7] Read More, Think More: Revisiting Observation Reduction for Web Agents. arXiv:2604.01535, 2026.

[8] Prune4Web: DOM Tree Pruning Programming for Web Agent. arXiv:2511.21398, 2025.

[9] Flash-Searcher: Fast and Effective Web Agents via DAG-Based Parallel Execution. arXiv:2509.25301, 2025.

[10] UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326, 2025.

[11] WebClipper: Efficient Evolution of Web Agents with Graph-based Trajectory Pruning. arXiv:2602.12852, 2026.